02 / ÁLGEBRA / CLASS_17_VECTORS / EX26

Guía de referencia Python

np.dot(), np.linalg.norm(), np.arccos(), np.degrees() y precisión numérica.

DOCUMENTO 02

Objetivos de aprendizaje

- Entender entradas, salidas y unidades de cada función.
- Calcular ángulos correctamente en radianes y grados.
- Identificar errores comunes antes de programar.

Conocimientos previos: Requisitos previos: EX25 completado, arrays NumPy, componentes y norma euclídea.

Orden de lectura: 01 Teoría → 02 Referencia → 03 Ejercicio → 04 Pseudocódigo → Programación.

1. Herramientas del EX26

La Guía Teórica explica qué es el producto escalar. Aquí nos centramos en **cómo expresarlo en Python**, qué recibe cada función y qué devuelve.

```
import numpy as np

u = np.array([1.0, 2.0])
v = np.array([3.0, 4.0])
```

2. np.dot(a, b)

Qué hace: para dos arrays unidimensionales de igual longitud, calcula su producto escalar y devuelve un único número.

Entradas: a, b = arrays numéricos con shape compatible; para este ejercicio, (2,).

Salida: escalar numérico, no array con dos componentes.

```
a = np.array([1.0, 2.0])
b = np.array([3.0, 4.0])
product = np.dot(a, b)
print(product)  # 11.0 = 1*3 + 2*4
```

Equivalencia para vectores 1D: a @ b. Lo estudiaremos también con matrices, donde el significado de @ depende de las dimensiones.

```
print(a @ b)      # 11.0
print(a * b)      # [3. 8.]  NO es producto escalar
```

3. np.linalg.norm(x)

Qué hace: para un vector 1D devuelve su norma euclídea por defecto. Es una magnitud escalar no negativa.

Entrada: x, array unidimensional. **Salida:** escalar float en estos ejemplos.

```
vector = np.array([5.0, 12.0])
length = np.linalg.norm(vector)
print(length)      # 13.0
```

Atención: np.linalg.norm([vector]) recibe una lista que contiene el vector y la interpreta como una matriz de una fila. Evita esos corchetes extra cuando la intención es medir un vector.

4. np.arccos(x)

Qué hace: calcula el ángulo cuyo coseno es x. **Entrada:** número entre −1 y 1. **Salida:** ángulo en **radianes**.

```
theta_rad = np.arccos(0.5)
print(theta_rad)   # about 1.0472 radians
```

5. np.degrees(x) y np.clip(x, low, high)

np.degrees(x): recibe un ángulo en radianes y devuelve su equivalente en grados.

```
degrees = np.degrees(theta_rad)
print(degrees)   # 60.0
```

np.clip(x, -1, 1): limita un número al intervalo [−1, 1]. Conviene usarlo sobre el coseno calculado, porque pequeñas imprecisiones pueden llevarlo a 1.0000000000000002.

```
cosine = 1.0000000000000002
safe_cosine = np.clip(cosine, -1.0, 1.0)
print(np.arccos(safe_cosine))  # 0.0
```

6. Cómo plantear un ángulo entre dos vectores

Sin entregar el algoritmo del EX26 resuelto: necesitamos el producto escalar, ambas normas, el cociente que da el coseno y finalmente arccos. Es necesario verificar que ninguna norma es cero antes de dividir.

$$c = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

$$\theta_{\text{deg}} = \text{degrees}(\arccos(c))$$

> No conviertas a grados antes de aplicar arccos. np.arccos() espera un coseno, no un ángulo. Si uno de los vectores es nulo, el ángulo no está definido; no utilices un pequeño número artificial en el denominador.

7. Comparaciones y formato de salida

Los cálculos con decimales pueden producir ligeras diferencias. Para comparar dos escalares cercanos podemos usar np.isclose; más adelante veremos tolerancias con detalle.

```
result = np.sqrt(2.0) ** 2
print(np.isclose(result, 2.0))  # True
print(f"Result: {result:.3f}")
```

8. Errores habituales y buenas prácticas

- Confundir np.dot(a, b) (escalar) con a*b (array).
- Usar ángulos en grados dentro de np.cos() sin convertirlos a radianes.
- Olvidar la comprobación de vectores nulos.
- Utilizar dimensiones incompatibles o datos sin unidades.
- Escribir el resultado esperado en el código como si fuera un valor calculado.

9. Hoja de consulta rápida

np.array([x, y]) — vector (2,)

np.dot(a,b) — scalar

a @ b — scalar for 1D vectors

np.linalg.norm(a) — length

np.arccos(c) — radians

np.degrees(angle) — degrees

np.clip(c,-1,1) — bounded cosine